

# E-Commerce Delivery & Shipping Data 2026

Exploratory Data Analysis (EDA) — Complete Code, Outputs, Graphs & Explanations

|                        |                                                        |
|------------------------|--------------------------------------------------------|
| <b>Dataset size</b>    | 50,000 rows × 30 columns                               |
| <b>Notebook cells</b>  | 31 cells                                               |
| <b>Analysis tools</b>  | Python • Pandas • NumPy • Matplotlib • Seaborn         |
| <b>Report contents</b> | Executed code • actual outputs • graphs • explanations |

**Important:** This report preserves the executed notebook's code and recorded outputs. The explanations are added to make the analysis easier to understand. No analytical result has been silently replaced with a newly calculated result.

# 1. Project Overview

This notebook performs an initial exploratory analysis of an e-commerce delivery and shipping dataset. The data contains order, customer, warehouse, product, delivery, payment, weather, rating, return, and tracking information. The notebook focuses on understanding the dataset, checking data quality, examining correlations, converting the order date, and visualizing delivery status, distance versus delay, shipping methods, payment methods, and delays by product category.

## Dataset structure observed in the notebook

|                            |                      |
|----------------------------|----------------------|
| order_id                   | order_date           |
| customer_id                | customer_segment     |
| customer_city              | customer_country     |
| warehouse_id               | warehouse_city       |
| product_category           | product_weight_kg    |
| order_value_usd            | shipping_method      |
| carrier                    | distance_km          |
| promised_delivery_days     | actual_delivery_days |
| delivery_status            | late_delivery        |
| delivery_delay_days        | shipping_cost_usd    |
| package_size               | payment_method       |
| order_priority             | weather_condition    |
| customer_rating            | return_requested     |
| return_reason              | delivery_attempts    |
| warehouse_processing_hours | tracking_status      |

## Cell 1 — Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

### Explanation

Imports the main libraries used in the project. Pandas is used for data handling, NumPy for numerical operations, Matplotlib and Seaborn for visualization.

Output: No recorded output.

## Cell 2 — Code

```
df = pd.read_csv('/content/drive/MyDrive/ANALYTICS/E-commerce_Delivery_Shipping_Data_2026.csv')
df
```

### Explanation

Reads the CSV file into a Pandas DataFrame named df and displays the complete dataset. The executed notebook shows 50,000 rows and 30 columns.

### Recorded output

```

order_id order_date customer_id customer_segment customer_city \
0 ORD-000001 2026-11-15 CUS-002353 Small Business Tokyo
1 ORD-000002 2026-04-08 CUS-003258 Consumer Vancouver
2 ORD-000003 2026-01-05 CUS-001221 Consumer Houston
3 ORD-000004 2026-05-29 CUS-001717 Small Business Delhi
4 ORD-000005 2026-02-22 CUS-000716 Small Business Nagoya
...
49995 ORD-049996 2026-09-05 CUS-004163 Small Business Leipzig
49996 ORD-049997 2026-12-07 CUS-004322 Consumer London
49997 ORD-049998 2026-11-14 CUS-000699 Enterprise San Diego
49998 ORD-049999 2026-09-02 CUS-004696 Consumer London
49999 ORD-050000 2026-04-11 CUS-000955 Consumer Sharjah

customer_country warehouse_id warehouse_city product_category \
0 Japan WH-004 Toronto Home & Kitchen
1 Canada WH-007 Paris Beauty
2 United States WH-005 London Home & Kitchen
3 India WH-002 Los Angeles Grocery
4 Japan WH-006 Berlin Beauty
...
49995 Germany WH-001 New York Electronics
49996 United Kingdom WH-005 London Toys
49997 United States WH-001 New York Automotive
49998 United Kingdom WH-007 Paris Books
49999 United Arab Emirates WH-001 New York Fashion

product_weight_kg ... package_size payment_method order_priority \
0 8.23 ... Large Digital Wallet Normal
1 0.10 ... Small Debit Card Normal
2 5.51 ... Large Debit Card Low
3 3.75 ... Large Debit Card Normal
4 0.10 ... Small Credit Card Normal
...
49995 0.86 ... Medium Digital Wallet High
49996 0.27 ... Small Cash on Delivery Normal
49997 7.76 ... Oversized Credit Card Low
49998 1.10 ... Medium PayPal Low
49999 1.36 ... Large Digital Wallet High

weather_condition customer_rating return_requested return_reason \
0 Snow 3 Yes Late Delivery
1 Cloudy 5 No NaN
2 Snow 4 Yes Product Defect
3 Snow 3 No NaN
4 Storm 2 No NaN
...
49995 Cloudy 4 No NaN
49996 Cloudy 4 No NaN
49997 Clear 3 Yes Late Delivery
49998 Snow 3 No NaN
49999 Cloudy 2 No NaN

delivery_attempts warehouse_processing_hours tracking_status
0 2 25.1 Delivered
1 1 25.9 Delivered
2 1 31.5 Delivered
3 1 12.6 In Transit
4 1 24.9 Delivered
...
49995 2 9.1 Delivered
49996 1 8.9 Delivered
49997 1 5.5 Delivered
49998 1 45.4 Delivered
49999 1 11.4 Delivered

[50000 rows x 30 columns]

```

### Cell 3 — Code

```
df.head()
```

#### Explanation

Displays the first five rows. This gives a quick look at the structure and the type of information stored in each record.

#### Recorded output

```

    order_id order_date customer_id customer_segment customer_city \
0  ORD-000001 2026-11-15 CUS-002353 Small Business Tokyo
1  ORD-000002 2026-04-08 CUS-003258 Consumer Vancouver
2  ORD-000003 2026-01-05 CUS-001221 Consumer Houston
3  ORD-000004 2026-05-29 CUS-001717 Small Business Delhi
4  ORD-000005 2026-02-22 CUS-000716 Small Business Nagoya

    customer_country warehouse_id warehouse_city product_category \
0      Japan      WH-004      Toronto Home & Kitchen
1      Canada      WH-007      Paris Beauty
2  United States      WH-005      London Home & Kitchen
3      India      WH-002      Los Angeles Grocery
4      Japan      WH-006      Berlin Beauty

    product_weight_kg ... package_size payment_method order_priority \
0      8.23 ... Large Digital Wallet Normal
1      0.10 ... Small Debit Card Normal
2      5.51 ... Large Debit Card Low
3      3.75 ... Large Debit Card Normal
4      0.10 ... Small Credit Card Normal

    weather_condition customer_rating return_requested return_reason \
0      Snow      3      Yes Late Delivery
1      Cloudy      5      No NaN
2      Snow      4      Yes Product Defect
3      Snow      3      No NaN
4      Storm      2      No NaN

    delivery_attempts warehouse_processing_hours tracking_status
0      2      25.1      Delivered
1      1      25.9      Delivered
2      1      31.5      Delivered
3      1      12.6      In Transit
4      1      24.9      Delivered

[5 rows x 30 columns]

```

## Cell 4 — Code

```
df.shape
```

### Explanation

Checks the dataset dimensions. The output is (50000, 30), meaning there are 50,000 rows and 30 columns.

### Recorded output

```
(50000, 30)
```

## Cell 5 — Code

```
df.columns
```

### Explanation

Lists all column names so the available variables can be identified before analysis and cleaning.

### Recorded output

```

Index(['order_id', 'order_date', 'customer_id', 'customer_segment',
      'customer_city', 'customer_country', 'warehouse_id', 'warehouse_city',
      'product_category', 'product_weight_kg', 'order_value_usd',
      'shipping_method', 'carrier', 'distance_km', 'promised_delivery_days',
      'actual_delivery_days', 'delivery_status', 'late_delivery',
      'delivery_delay_days', 'shipping_cost_usd', 'package_size',
      'payment_method', 'order_priority', 'weather_condition',
      'customer_rating', 'return_requested', 'return_reason',
      'delivery_attempts', 'warehouse_processing_hours', 'tracking_status'],
      dtype='object')

```

## Cell 6 — Code

```
df.info()
```

### Explanation

Checks the DataFrame structure, non-null counts, and data types. The notebook output shows 20 object columns, 5 float64 columns, 5 int64 columns, and one column (return\_reason) with missing values.

### Recorded output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 30 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   order_id                             50000 non-null  object
1   order_date                           50000 non-null  object
2   customer_id                          50000 non-null  object
3   customer_segment                     50000 non-null  object
4   customer_city                        50000 non-null  object
5   customer_country                     50000 non-null  object
6   warehouse_id                         50000 non-null  object
7   warehouse_city                       50000 non-null  object
8   product_category                     50000 non-null  object
9   product_weight_kg                    50000 non-null  float64
10  order_value_usd                       50000 non-null  float64
11  shipping_method                       50000 non-null  object
12  carrier                              50000 non-null  object
13  distance_km                           50000 non-null  float64
14  promised_delivery_days                50000 non-null  int64
15  actual_delivery_days                  50000 non-null  int64
16  delivery_status                       50000 non-null  object
17  late_delivery                         50000 non-null  object
18  delivery_delay_days                   50000 non-null  int64
19  shipping_cost_usd                     50000 non-null  float64
20  package_size                          50000 non-null  object
21  payment_method                        50000 non-null  object
22  order_priority                        50000 non-null  object
23  weather_condition                     50000 non-null  object
24  customer_rating                       50000 non-null  int64
25  return_requested                       50000 non-null  object
26  return_reason                         8625 non-null   object
27  delivery_attempts                     50000 non-null  int64
28  warehouse_processing_hours            50000 non-null  float64
29  tracking_status                       50000 non-null  object
dtypes: float64(5), int64(5), object(20)
memory usage: 11.4+ MB
```

### Cell 7 — Code

```
df.describe(include='all')
```

#### Explanation

Generates descriptive statistics for all columns, including numerical summaries and categorical-column information.

#### Recorded output

```

count      order_id  order_date  customer_id  customer_segment  customer_city  \
unique      50000      365      6348      4      99
top      ORD-049984  2026-06-18  CUS-006249      Consumer      Singapore
freq      1      172      22      30167      1568
mean      NaN      NaN      NaN      NaN      NaN
std      NaN      NaN      NaN      NaN      NaN
min      NaN      NaN      NaN      NaN      NaN
25%      NaN      NaN      NaN      NaN      NaN
50%      NaN      NaN      NaN      NaN      NaN
75%      NaN      NaN      NaN      NaN      NaN
max      NaN      NaN      NaN      NaN      NaN

count      customer_country  warehouse_id  warehouse_city  product_category  \
unique      17      10      10      12
top      United States      WH-001      New York      Electronics
freq      12535      7597      7597      7406
mean      NaN      NaN      NaN      NaN
std      NaN      NaN      NaN      NaN
min      NaN      NaN      NaN      NaN
25%      NaN      NaN      NaN      NaN
50%      NaN      NaN      NaN      NaN
75%      NaN      NaN      NaN      NaN
max      NaN      NaN      NaN      NaN

count      product_weight_kg  ...  package_size  payment_method  order_priority  \
unique      NaN      ...      4      6      4
top      NaN      ...      Medium      Credit Card      Normal
freq      NaN      ...      18648      15032      22450
mean      2.255028      ...      NaN      NaN      NaN
std      3.694952      ...      NaN      NaN      NaN
min      0.100000      ...      NaN      NaN      NaN
25%      0.320000      ...      NaN      NaN      NaN
50%      0.930000      ...      NaN      NaN      NaN
75%      2.610000      ...      NaN      NaN      NaN
max      52.190000      ...      NaN      NaN      NaN

count      weather_condition  customer_rating  return_requested  return_reason  \
unique      6      NaN      2      6
top      Clear      NaN      No      Product Defect
freq      14932      NaN      41375      2184
mean      NaN      3.371840      NaN      NaN
std      NaN      0.998777      NaN      NaN
min      NaN      1.000000      NaN      NaN
25%      NaN      3.000000      NaN      NaN
50%      NaN      3.000000      NaN      NaN
75%      NaN      4.000000      NaN      NaN
max      NaN      5.000000      NaN      NaN

count      delivery_attempts  warehouse_processing_hours  tracking_status
unique      NaN      NaN      4
top      NaN      NaN      Delivered
freq      NaN      NaN      44184
mean      1.169860      14.412430      NaN
std      0.390756      9.924904      NaN
min      1.000000      1.000000      NaN
25%      1.000000      6.600000      NaN
50%      1.000000      11.900000      NaN
75%      1.000000      20.700000      NaN
max      3.000000      48.000000      NaN

```

[11 rows x 30 columns]

## Checking for missing values

Section heading introducing the missing-value check.

### Cell 9 — Code

```
df.isnull().sum()
```

#### Explanation

Counts missing values in every column. This helps identify fields that need attention before further analysis.

#### Recorded output

```
order_id          0
order_date        0
customer_id       0
customer_segment  0
customer_city     0
customer_country  0
warehouse_id      0
warehouse_city    0
product_category  0
product_weight_kg 0
order_value_usd   0
shipping_method   0
carrier           0
distance_km       0
promised_delivery_days 0
actual_delivery_days 0
delivery_status   0
late_delivery     0
delivery_delay_days 0
shipping_cost_usd 0
package_size      0
payment_method    0
order_priority    0
weather_condition 0
customer_rating   0
return_requested  0
return_reason     41375
delivery_attempts 0
warehouse_processing_hours 0
tracking_status   0
dtype: int64
```

## Finding the correlation for the numeric values

Section heading introducing correlation analysis for numerical variables.

### Cell 11 — Code

```
corr=df.corr(numeric_only=True)
corr
```

#### Explanation

Calculates the correlation matrix using only numeric columns and displays it. Correlation values help identify linear relationships between numerical variables.

#### Recorded output

|                            | product_weight_kg          | order_value_usd        | distance_km \ |
|----------------------------|----------------------------|------------------------|---------------|
| product_weight_kg          | 1.000000                   | 0.145075               | 0.000468      |
| order_value_usd            | 0.145075                   | 1.000000               | 0.002980      |
| distance_km                | 0.000468                   | 0.002980               | 1.000000      |
|                            | promised_delivery_days     | actual_delivery_days \ |               |
| promised_delivery_days     | 0.003142                   | 0.004399               | 0.485015      |
| actual_delivery_days       | 0.003014                   | 0.005737               | 0.519697      |
| delivery_delay_days        | 0.001145                   | 0.005833               | 0.361780      |
|                            | shipping_cost_usd          | customer_rating        |               |
| shipping_cost_usd          | 0.238476                   | 0.049036               | 0.817135      |
| customer_rating            | -0.003008                  | -0.005886              | -0.223610     |
| delivery_attempts          | -0.008022                  | 0.004670               | 0.009119      |
| warehouse_processing_hours | -0.002115                  | 0.001447               | 0.001705      |
|                            | promised_delivery_days     | actual_delivery_days \ |               |
| product_weight_kg          | 0.003142                   | 0.003014               |               |
| order_value_usd            | 0.004399                   | 0.005737               |               |
| distance_km                | 0.485015                   | 0.519697               |               |
| promised_delivery_days     | 1.000000                   | 0.929923               |               |
| actual_delivery_days       | 0.929923                   | 1.000000               |               |
| delivery_delay_days        | 0.416878                   | 0.713783               |               |
| shipping_cost_usd          | 0.514991                   | 0.534442               |               |
| customer_rating            | -0.195267                  | -0.362743              |               |
| delivery_attempts          | -0.000307                  | 0.020894               |               |
| warehouse_processing_hours | 0.007261                   | 0.108794               |               |
|                            | delivery_delay_days        | shipping_cost_usd \    |               |
| product_weight_kg          | 0.001145                   | 0.238476               |               |
| order_value_usd            | 0.005833                   | 0.049036               |               |
| distance_km                | 0.361780                   | 0.817135               |               |
| promised_delivery_days     | 0.416878                   | 0.514991               |               |
| actual_delivery_days       | 0.713783                   | 0.534442               |               |
| delivery_delay_days        | 1.000000                   | 0.343278               |               |
| shipping_cost_usd          | 0.343278                   | 1.000000               |               |
| customer_rating            | -0.507987                  | -0.182028              |               |
| delivery_attempts          | 0.054422                   | 0.004269               |               |
| warehouse_processing_hours | 0.237527                   | -0.000613              |               |
|                            | customer_rating            | delivery_attempts \    |               |
| product_weight_kg          | -0.003008                  | -0.008022              |               |
| order_value_usd            | -0.005886                  | 0.004670               |               |
| distance_km                | -0.223610                  | 0.009119               |               |
| promised_delivery_days     | -0.195267                  | -0.000307              |               |
| actual_delivery_days       | -0.362743                  | 0.020894               |               |
| delivery_delay_days        | -0.507987                  | 0.054422               |               |
| shipping_cost_usd          | -0.182028                  | 0.004269               |               |
| customer_rating            | 1.000000                   | -0.044638              |               |
| delivery_attempts          | -0.044638                  | 1.000000               |               |
| warehouse_processing_hours | -0.186202                  | 0.020589               |               |
|                            | warehouse_processing_hours |                        |               |
| product_weight_kg          | -0.002115                  |                        |               |
| order_value_usd            | 0.001447                   |                        |               |
| distance_km                | 0.001705                   |                        |               |
| promised_delivery_days     | 0.007261                   |                        |               |
| actual_delivery_days       | 0.108794                   |                        |               |
| delivery_delay_days        | 0.237527                   |                        |               |
| shipping_cost_usd          | -0.000613                  |                        |               |
| customer_rating            | -0.186202                  |                        |               |
| delivery_attempts          | 0.020589                   |                        |               |
| warehouse_processing_hours | 1.000000                   |                        |               |

## visualizing the corelation

Section heading introducing the visualization of the correlation matrix.

### Cell 13 — Code

```
sns.heatmap(corr)
```

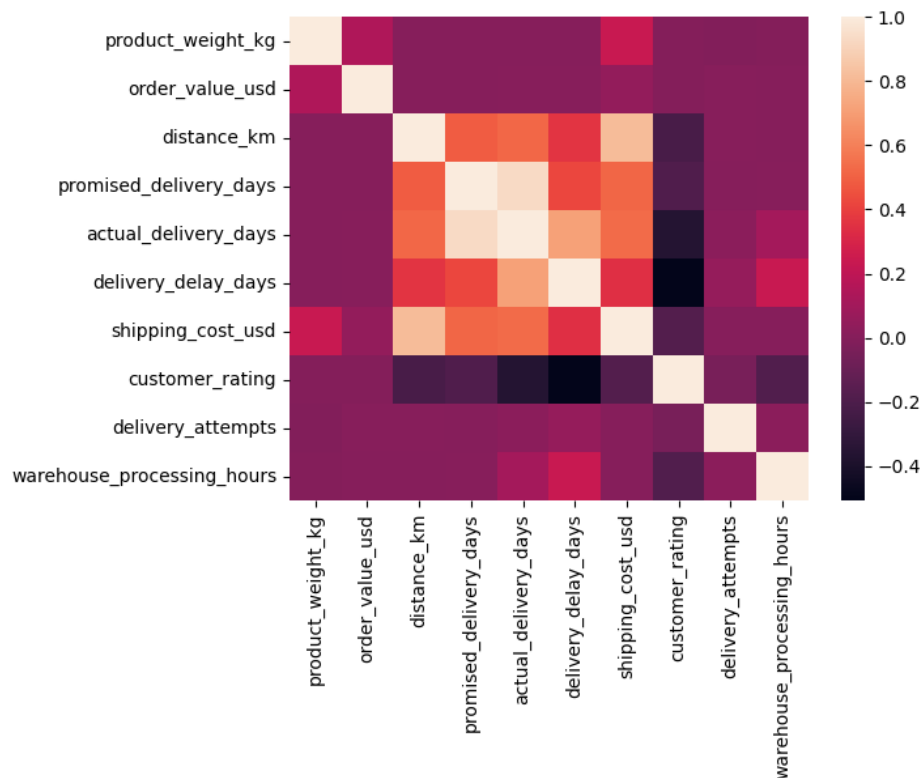
### Explanation

Creates a heatmap of the correlation matrix. The visual makes stronger positive or negative relationships easier to spot.

### Recorded output

```
<Axes: >
```

### Graph / visual output



### Cell 14 — Code

```
df.duplicated().sum()
```

#### Explanation

Counts completely duplicated rows. A result of zero means no exact duplicate rows were found.

#### Recorded output

```
np.int64(0)
```

## Converting order date

Section heading for converting the order\_date column to a proper date/time type.

### Cell 16 — Code

```
df['order_date']=pd.to_datetime(df['order_date'])
```

#### Explanation

Converts order\_date from text/object format into Pandas datetime format, enabling reliable date-based analysis.

Output: No recorded output.

### Cell 17 — Code

```
df['order_date'].min()
df['order_date'].max()
```

#### Explanation

Checks the earliest and latest order dates. In the executed notebook, only the final expression is displayed because the two expressions were entered in the same code cell.

#### Recorded output

```
Timestamp('2026-12-31 00:00:00')
```

## Cell 18 — Code

```
# Empty code cell
```

### Explanation

Empty code cell retained from the original notebook.

Output: No recorded output.

## Visualization of Delivery Status Distribution

Section heading for delivery-status visualization.

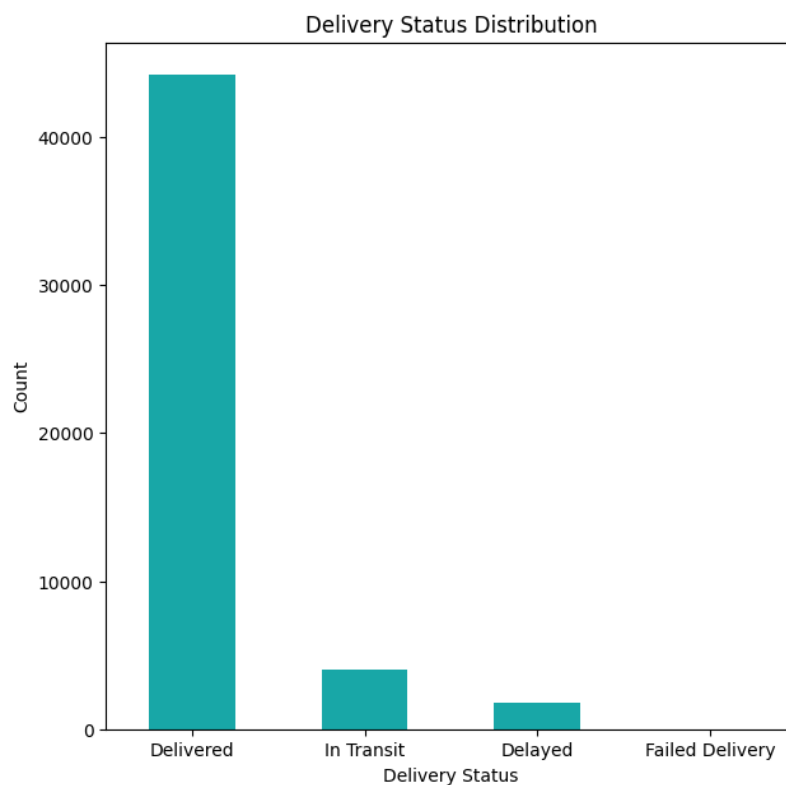
## Cell 20 — Code

```
plt.figure(figsize=(7, 7))
sns.countplot(data=df, x='delivery_status', color='c', width=0.5)
plt.title('Delivery Status Distribution')
plt.xlabel('Delivery Status')
plt.ylabel('Count')
plt.show()
```

### Explanation

Creates a count plot showing how many orders fall into each delivery-status category.

### Graph / visual output



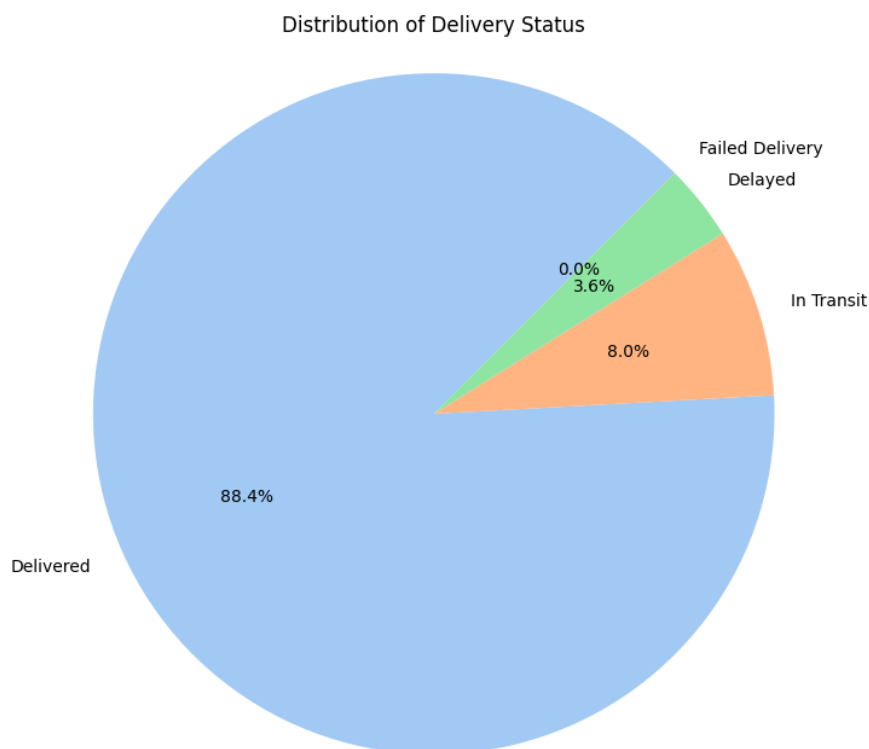
## Cell 21 — Code

```
plt.figure(figsize=(8,8 ))
delivery_status_counts = df['delivery_status'].value_counts()
plt.pie(delivery_status_counts, labels=delivery_status_counts.index, autopct='%1.1f%%', startangle=45,
        colors=sns.color_palette('pastel'))
plt.title('Distribution of Delivery Status')
plt.axis('equal')
plt.show()
```

### Explanation

Creates a pie chart showing the proportion of orders in each delivery-status category.

### Graph / visual output



### Cell 22 — Code

```
df['delivery_status'].value_counts()
```

### Explanation

Counts the number of orders in each delivery-status category and prints the totals.

### Recorded output

```
delivery_status
Delivered      44184
In Transit     4007
Delayed        1806
Failed Delivery    3
Name: count, dtype: int64
```

## Visualization of Distance vs Delivery Delays

Section heading for examining distance and delivery delays.

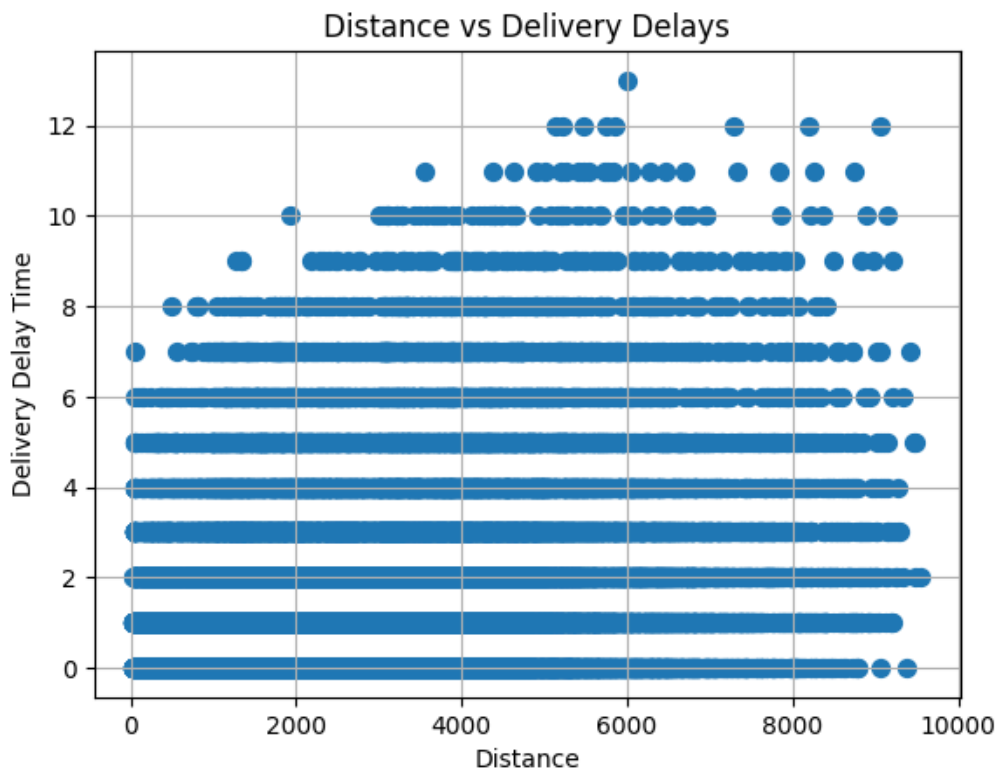
### Cell 24 — Code

```
plt.scatter(df['distance_km'], df['delivery_delay_days'],s=50)
plt.xlabel('Distance')
plt.ylabel('Delivery Delay Time')
plt.title('Distance vs Delivery Delays')
plt.grid()
plt.show()
```

### Explanation

Creates a scatter plot of distance versus delivery delay days to visually examine whether longer distances are associated with larger delays.

### Graph / visual output



## Commonly used shipping method

Section heading for identifying commonly used shipping methods.

### Cell 26 — Code

```
df['shipping_method'].value_counts().plot(kind='bar',color='g')
plt.title('Orders by shipping methods')
plt.xlabel('Shipping Method')
plt.ylabel('Number of oredrs')
plt.show()
```

### Explanation

Counts orders by shipping method and displays the result as a bar chart.

### Graph / visual output



## Common payment methods customers used

Section heading for examining customer payment methods.

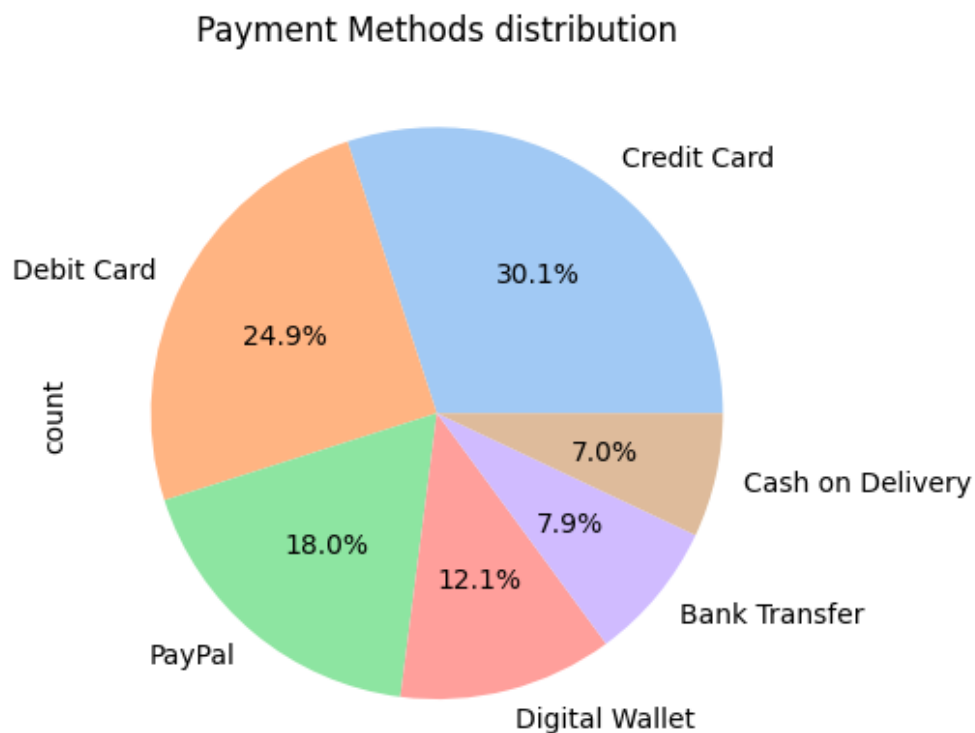
### Cell 28 — Code

```
df['payment_method'].value_counts().plot(kind='pie', autopct='%1.1f%%', colors=sns.color_palette('pastel'))  
plt.title('Payment Methods distribution')  
plt.show()
```

### Explanation

Counts payment methods and displays their proportions as a pie chart.

### Graph / visual output



## Product categories with highest delays

Section heading for comparing delivery delays across product categories.

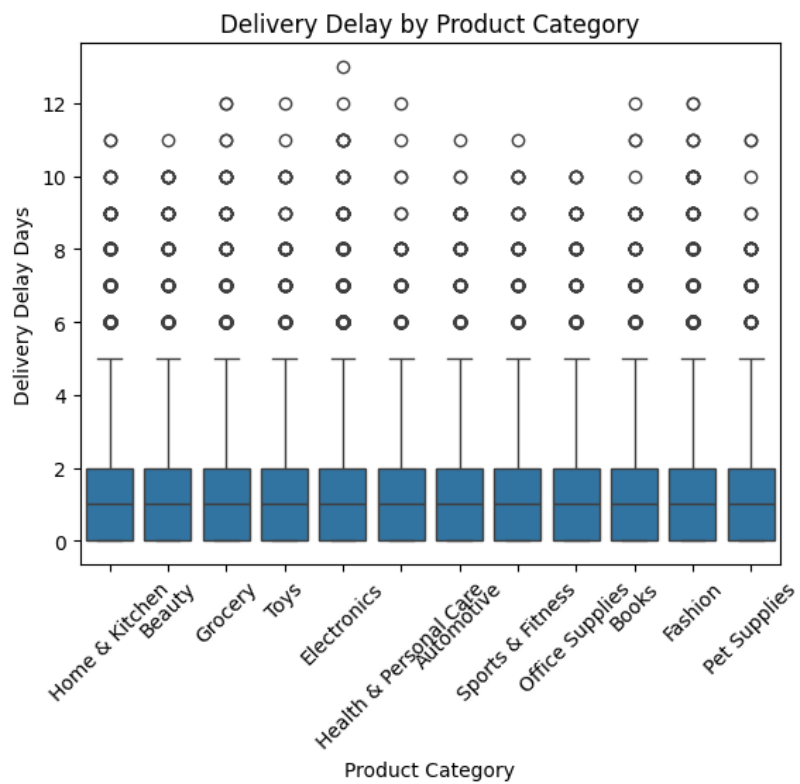
### Cell 30 — Code

```
sns.boxplot(data=df, x='product_category', y='delivery_delay_days')
plt.title('Delivery Delay by Product Category')
plt.xlabel('Product Category')
plt.ylabel('Delivery Delay Days')
plt.xticks(rotation=45)
plt.show()
```

### Explanation

Uses a boxplot to compare the distribution of delivery-delay days across product categories, including median, spread, and potential outliers.

### Graph / visual output



### Cell 31 — Code

```
# Empty code cell
```

#### Explanation

Empty code cell retained from the original notebook.

Output: No recorded output.